

Lecture 20: TCP Part 2

Congestion Control and the Modern Picture

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Explain why TCP congestion control is necessary and differs from flow control
- Describe slow start, congestion avoidance (AIMD), and fast recovery
- Trace **cwnd** and **ssthresh** evolution in response to timeout and triple dup ACK
- State the TCP throughput formula and explain its RTT/loss-rate dependence
- Explain why high-BDP paths need window scaling and why Reno fails there
- Describe CUBIC's key departure from Reno
- Explain ECN and why it is preferable to loss-based signals
- Describe the AIMD fairness argument
- Explain why QUIC was designed and what TCP limitations it addresses

Key Acronyms

Acronym	Meaning	Acronym	Meaning
ACK	Acknowledgment	HOL	Head-of-Line (blocking)
AIMD	Additive Increase Multiplicative Decrease	MSS	Maximum Segment Size
BDP	Bandwidth-Delay Product	RTT	Round-Trip Time
CA	Congestion Avoidance	RTO	Retransmission Timeout
cwnd	Congestion Window	ssthresh	Slow-Start Threshold
ECN	Explicit Congestion Notification	QUIC	Quick UDP Internet Connections

L19 left one variable undefined: `cwnd`

Limit	Set by	Protects
<code>rwnd</code>	Receiver	Receiver's buffer
<code>cwnd</code>	Sender	Network from congestion
Effective window	$\min(\text{cwnd}, \text{rwnd})$ — both	

- **Flow control** is *explicit*: receiver tells sender its available buffer via `rwnd`
- **Congestion control** is *implicit*: sender infers network state from loss signals
- No router ever says “I am congested” — TCP must infer it from dropped packets.

Why Congestion Control?

- **Congestion collapse:** aggregate traffic exceeds capacity $\Rightarrow$ queues fill $\Rightarrow$ drops $\Rightarrow$ retransmits $\Rightarrow$ even more traffic $\Rightarrow$ collapse
- **TCP's contract:** probe aggressively for bandwidth; back off immediately and sharply on loss
- Without self-regulation: greedy senders starve all others
- TCP Reno uses loss as the congestion signal:
 - **Timeout** — severe signal (nothing getting through)
 - **3 duplicate ACKs** — mild signal (later packets still arriving)

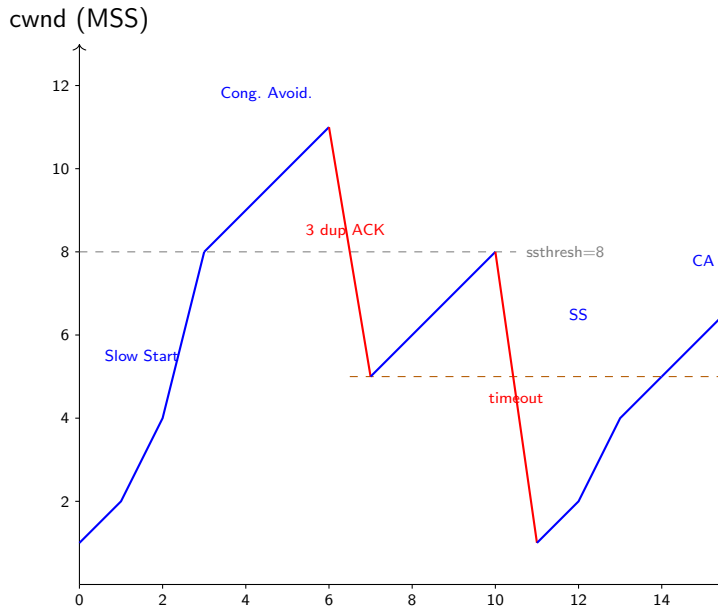
"No router ever says 'I am congested' — TCP must infer it."

- **cwnd** starts at **1 MSS**; increases by 1 MSS per ACK received $\Rightarrow$ doubles every RTT (exponential growth)
- “Slow” refers to the starting point (1 MSS), *not* the growth rate
- **ssthresh** controls when slow start stops:
 - $\text{cwnd} < \text{ssthresh}$: slow start (exponential)
 - $\text{cwnd} \geq \text{ssthresh}$: congestion avoidance (linear, +1 MSS/RTT)
- **ssthresh** initialized large; set to $\text{cwnd}/2$ on first loss

RTT	cwnd (MSS)
0	1
1	2
2	4
3	8
4	16

Each ACK $\Rightarrow$ +1 MSS
Each RTT $\Rightarrow$ cwnd $\times 2$

Congestion Avoidance (AIMD) and the Sawtooth



Additive Increase (CA phase):

$$cwnd \leftarrow cwnd + \frac{MSS^2}{cwnd} \text{ per ACK}$$

When **3 duplicate ACKs** arrive: fast retransmit + fast recovery

- 1 $\text{ssthresh} = \text{cwnd} / 2$
- 2 $\text{cwnd} = \text{ssthresh}$ (*skip slow start — dup ACKs prove later packets flow*)
- 3 Retransmit the missing segment immediately
- 4 Each *additional* dup ACK while in recovery: $\text{cwnd} += 1 \text{ MSS}$ (each dup ACK = one more segment delivered to receiver)
- 5 New ACK arrives: $\text{cwnd} = \text{ssthresh} \Rightarrow$ return to congestion avoidance

Fast recovery (3 dup ACKs):

Network partly working; reduce cwnd by half, stay in CA

Timeout:

Nothing getting through; $\text{cwnd} \rightarrow 1$, full slow start

Worked cwnd Trace

Event	cwnd before	Action	cwnd after	ssthresh
Start	—	SS begins	1	8
ACK (SS)	1	$\times 2/\text{RTT}$	2	8
ACK (SS)	2	$\times 2/\text{RTT}$	4	8
ACK (SS)	4	$\times 2/\text{RTT}$	8	8
ACK (CA)	8	$+1/\text{RTT}$	9	8
ACK (CA)	9	$+1/\text{RTT}$	10	8
3 dup ACKs	10	ssthresh=5, cwnd=5	5	5
New ACK	5	CA resumes	6	5
timeout	8	ssthresh=4, cwnd=1	1	4
SS resumes	1	exponential $\rightarrow$ 4	...	4

At **ssthresh** the mode switches: slow start (exponential) $\rightarrow$ congestion avoidance (linear).
Timeout resets cwnd to 1; 3 dup ACKs halve cwnd (much gentler).

Reading TCP internals directly from the kernel:

```
$ ss -ti dst 93.184.216.34
tcp ESTAB 0 0 192.168.1.5:45678 93.184.216.34:443
    cubic wscale:7,7 rto:204 rtt:4.5/1.1 ato:40 mss:1448
    cwnd:10 ssthresh:10
    send 257.6Mb/s lastsnd:8 lastrcv:8 lastack:8
    app_limited:1
```

cubic	Congestion control algorithm (Linux default since kernel 2.6.19)
rtt:4.5/1.1	SRTT = 4.5 ms, RTTVAR = 1.1 ms (Jacobson estimator from L19)
rto:204	RTO formula ≈ 9 ms; RFC 6298 floor = 200 ms — floor is active
cwnd:10 ssthresh:10	At the CA/SS boundary
app_limited:1	Application, not network, is the bottleneck here

TCP Throughput Formula

Mathis et al. (1997) steady-state approximation

$$BW \approx \frac{MSS}{RTT} \cdot \frac{1}{\sqrt{p}}$$

- **Intuition:** sawtooth between $W/2$ and W ; average cwnd $\approx 3W/4$; loss rate $p \approx 2/(3W^2)$
- Throughput $\propto 1/RTT$ — longer paths get systematically less bandwidth
- Throughput $\propto 1/\sqrt{p}$ — loss is very costly

Striking implication

To achieve **10 Gb/s** over $RTT = 100$ ms with $MSS = 1460$ B, the required loss rate is $p < 10^{-10}$ — fewer than one dropped segment per 10 billion sent. In practice, high-speed TCP needs *near-zero* loss.

Bandwidth-Delay Product and Window Scaling

BDP = $BW \times RTT$ = “pipe capacity” in bytes

Problem: TCP Window field = 16 bits $\Rightarrow$ max **64 KB** without options

Example: $1 \text{ Gb/s} \times 100 \text{ ms} = 12.5 \text{ MB}$ needed; $\frac{64 \text{ KB}}{0.1 \text{ s}} = 5.2 \text{ Mb/s}$ ($< 1\%$ of link!)

Window Scale option (RFC 7323): multiplies window by 2^n , $n \in [0, 14] \Rightarrow$ up to 1 GB

BW	RTT	BDP	Required wscale
100 Mb/s	10 ms	125 KB	1 ($\times 2$)
1 Gb/s	100 ms	12.5 MB	8 ($\times 256$)
10 Gb/s	100 ms	125 MB	12 ($\times 4096$)

TCP Reno vs. TCP CUBIC

	TCP Reno	TCP CUBIC
Growth clock	RTT-clocked	Wall-clock time
Growth shape	Linear (AIMD)	Cubic function of time since last loss
High-BDP behavior	Slow recovery	Fast recovery (large W far from last loss)
RTT fairness	Short RTT flows get more	RTT-independent growth
Linux default	No (superseded)	Yes (since kernel 2.6.19)

CUBIC key idea

$W(t) = C(t - K)^3 + W_{\max}$ where K is chosen so that $W(0) = W_{\max}/2$. Growth is slow near the last loss point (probing cautiously) and fast when far from it (filling available bandwidth quickly).

Note: CUBIC is the default congestion control in Linux, macOS, and most modern OSes.

Loss-based signaling (today's default):

- 1 Router's queue fills
- 2 Router drops packet
- 3 ACK gap detected by sender
- 4 `cwnd` reduced — *after* losing goodput

ECN (RFC 3168):

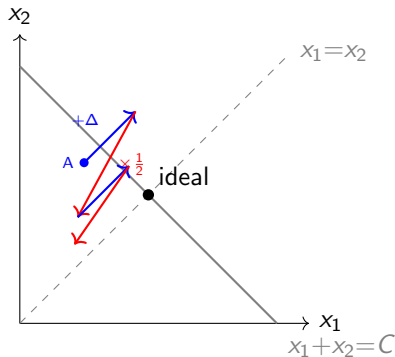
- 1 Router's queue *begins to build*
- 2 Router **marks** IP header (CE bit) — packet delivered!
- 3 Receiver echoes CE via **ECE** flag in TCP ACK
- 4 Sender reduces `cwnd` as if loss occurred

Benefits

- No goodput lost from the dropped packet
- Earlier signal (queue building, not yet overflowing)
- Lower latency under load

Caveat

Requires both endpoints *and* all routers to support ECN. Widely deployed in datacenters; less universal on the public Internet.



Two flows sharing capacity C :

- **Additive increase:** both grow by Δ per RTT — move *parallel* to fairness line
- **Multiplicative decrease:** both halve — move toward *origin*
- Net effect: spiral toward equal-share, full-utilization point

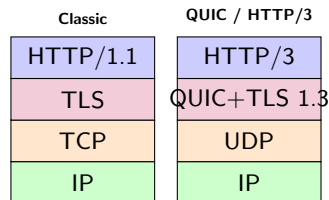
- **RTT unfairness:** TCP Reno gives more bandwidth to shorter-RTT flows (more AIMD steps per second). CUBIC partially fixes this — wall-clock growth is RTT-independent.
- **UDP unfriendliness:** UDP flows do not participate in AIMD. A greedy UDP sender will crowd out TCP. “Friendly” protocols (DCCP, QUIC) implement their own rate control.
- **Multiple connections:** a client opening N TCP connections to a server captures N times the AIMD share. HTTP/1.1 browsers exploited this with 6 parallel connections per host.

What TCP Gets Wrong (and Why It's Hard to Fix)

- ❶ **HOL blocking in multiplexed streams** — HTTP/2 runs many streams over one TCP byte-stream; one lost packet *blocks all streams* until retransmit. QUIC delivers each stream independently.
- ❷ **Slow connection setup** — TCP 3WSH + TLS handshake = 2–3 RTTs before data flows. QUIC + TLS 1.3: **1 RTT** new connection, **0 RTT** resumed.
- ❸ **Ossification** — Middleboxes inspect and modify TCP headers; changing TCP semantics is nearly impossible in practice. QUIC header is *fully encrypted* (over UDP); middleboxes cannot see or tamper with it.
- ❹ **No connection migration** — TCP is identified by a 4-tuple; switching networks (e.g., WiFi → cellular) breaks the connection. QUIC uses a **connection ID** independent of addresses.

The fix requires updating all deployed middleboxes — effectively impossible. QUIC bypasses the problem by running over UDP.

QUIC — What It Is



QUIC replaces TCP + TLS for web traffic:

- Transport protocol over UDP
- Mandatory built-in TLS 1.3
- Multiple *independent* streams (no HOL blocking)
- Own congestion control (CUBIC or BBR)
- Used by: HTTP/3, YouTube, Google Search
- $\approx 25\%$ of Internet traffic (2024)

UDP / TCP / QUIC at a Glance

Feature	UDP	TCP	QUIC
Connection setup	None	3-way handshake (1 RTT)	1 RTT (0 RTT resumed)
Reliability	None	Yes (cumul. ACK + retx)	Yes (per stream)
Ordering	None	Yes (byte stream)	Yes (per stream)
Flow control	None	rwnd	Yes
Congestion control	None	AIMD (cwnd)	CUBIC / BBR
HOL blocking	N/A	Yes (one byte stream)	No (indep. streams)
Encryption	None (app layer)	TLS separate	TLS 1.3 built-in
Header size	8 bytes	20+ bytes	Variable (encrypted)
Runs over	IP	IP	UDP
Typical use	DNS, streaming, games	Web, SSH, bulk transfer	HTTP/3, YouTube

QUIC combines the low overhead of UDP with the reliability of TCP and fixes TCP's structural limitations.

The Socket Programmer's View

The application sees: a connected socket — reliable, ordered, full-duplex byte stream. All TCP machinery (RTT estimation, cwnd, retransmits) is *invisible to the app*.

`send()` writes bytes in; `recv()` reads bytes out.

Socket Option	Effect
TCP_NODELAY	Disable Nagle's algorithm; send immediately (low latency)
TCP_INFO (Linux)	Read cwnd, ssthresh, RTT, retransmits from kernel
SO_RCVBUF / SO_SNDBUF	Tune socket buffer sizes; affects rwnd and throughput

“This is what you will use in L24 (sockets lab).”

Transport Layer Summary — L18–L20

Concept	Where	Key idea
Multiplexing / demux	L18	Port numbers identify processes
UDP	L18	Best-effort, 8-byte header, app controls everything
Stop-and-Wait / GBN / SR	L18	Pipeline mechanisms; S&W utilization, window fills pipe
TCP segment header	L19	20-byte min; seq/ack are byte offsets
3-way handshake	L19	SYN/SYN-ACK/ACK; ISN randomization
Reliable delivery + RTO	L19	EWMA + RTTVAR; Karn's algorithm; backoff
Flow control (rwnd)	L19	Receiver-driven; prevents buffer overflow
Slow start / AIMD	L20	Probe then back off; cwnd doubles, then $+1/\text{RTT}$
Fast recovery	L20	3 dup ACKs $\rightarrow$ skip slow start
Congestion window	L20	Effective window = $\min(\text{cwnd}, \text{rwnd})$
QUIC	L20	UDP-based; fixes HOL, ossification, migration

Discussion Questions

- ➊ Why does TCP Reno perform poorly on satellite links (high RTT, high BDP) even at low loss rates?
- ➋ Two TCP flows share a 100 Mb/s link: one has $RTT = 10$ ms, the other $RTT = 100$ ms. Do they share the bandwidth equally under Reno? How does CUBIC improve this?
- ➌ If a router uses ECN instead of drops, what happens to the TCP throughput formula? Does p still measure the same thing?
- ➍ A UDP video application has no rate control. What happens to co-existing TCP flows? Is this fair?
- ➎ HTTP/2 claims to solve HOL blocking with multiplexing. Why does QUIC claim HTTP/2 over TCP *still* has HOL blocking?
- ➏ Why must QUIC implement its own congestion control rather than relying on the OS TCP stack?